

Comprendre ton projet. Le présenter avec des preuves.

Un seul conducteur de 15 minutes, les explications de chaque brique, les gestes de démonstration et les réponses aux questions de Jérémy.

État vérifié aujourd'hui

76 tests passent ; Docker et Swagger fonctionnent. La démo directe a été rétablie avec minstral-8b-2512. Les scores de juillet concernent l'ancien modèle. La démo de secours est explicitement enregistrée. Le corpus couvre juillet 2025 → juillet 2026 ; il ne représente pas un agenda actualisé pour aujourd'hui.

Un assistant culturel fondé sur des événements réels

L'idée à retenir : Un POC complet pour tester la recherche en langage naturel.

Bonjour Jérémy. Je vais vous présenter le prototype réalisé pour Puls-Events, un assistant de recommandation culturelle qui utilise les événements OpenAgenda. Le besoin est de permettre aux utilisateurs de poser une question naturellement, puis de recevoir une réponse qu'ils peuvent vérifier grâce aux sources. Mon livrable couvre toute la chaîne : collecte et préparation des données, recherche vectorielle, génération de réponse, API et conteneur Docker. Le corpus livré contient 7 586 événements parisiens. Il s'agit d'un POC, donc d'une preuve de faisabilité. Je vais expliquer les choix, montrer l'API et présenter les résultats ainsi que les limites avant un déploiement plus large.

À montrer / à faire

Partage uniquement la fenêtre du diaporama. Garde ce guide hors du partage.

Transition à dire : « Commençons par le risque métier que le RAG cherche à réduire. »

Le RAG relie la réponse au catalogue

L'idée à retenir : Rechercher les informations, puis rédiger à partir de ces informations.

Le problème métier est qu'une recherche par mots-clés ne correspond pas toujours à la façon dont un utilisateur exprime son envie. Par exemple, une sortie en famille peut correspondre à une visite jouée par des comédiens, même si les mots utilisés diffèrent. Un modèle de langage seul peut produire une réponse convaincante sans connaître les événements du catalogue. Le RAG combine deux capacités. Il recherche d'abord des passages proches du sens de la question. Il ajoute ensuite ces passages au contexte du modèle. Enfin, Mistral formule une réponse en français. Les sources permettent de revenir aux événements d'origine. Le RAG réduit ainsi le risque d'invention, mais ne le supprime pas : il faut encore vérifier la pertinence des passages et la fidélité de la réponse.

À montrer / à faire

Pointer les trois étapes. L'exemple famille existe dans le jeu RAGAS ; ce n'est pas une nouvelle preuve live.

Transition à dire : « Cette approche dépend d'abord de la qualité du catalogue préparé. »

Le corpus est un instantané explicite de Paris

L'idée à retenir : Paris · du 10 juillet 2025 au 10 juillet 2026 · 7 586 événements.

Les données proviennent d'OpenAgenda à travers le jeu public exposé par Opendatasoft. Cette source est celle du lien donné dans la mission et ne demande pas de clé pour la collecte. Le script filtre Paris et la première date des événements, du 10 juillet 2025 au 10 juillet 2026. Il parcourt les résultats, nettoie les descriptions HTML, gère les champs absents et supprime les doublons par identifiant. Il produit un fichier Parquet structuré et un manifeste qui trace les paramètres et le volume. Je conserve les dates, le lieu et l'URL pour que la réponse soit vérifiable. Cette fenêtre était celle du POC construit en juillet. Pour une utilisation aujourd'hui, le corpus doit être actualisé : je ne présente pas ce catalogue figé comme un agenda à jour.

À montrer / à faire

Explique la fenêtre telle qu'elle est. Si on te demande le fichier, ouvre le manifeste de collecte, pas le JSON brut entier.

Transition à dire : « Une fois les fiches préparées, il faut les rendre recherchables par leur sens. »

03:05 → 04:20 • 75 secondes • repère incluant gestes et pauses

Les descriptions deviennent des passages recherchables

L'idée à retenir : 1 000 caractères par passage, 200 de chevauchement, vecteurs de 1 024 dimensions.

Certaines fiches sont longues. Les indexer comme un seul bloc risquerait de diluer une information utile. J'utilise donc le découpage récursif de LangChain, configuré à 1 000 caractères avec un chevauchement maximal de 200 caractères. Le chevauchement limite la perte d'information aux frontières. Par exemple, une fiche CHIMERE contient 10 273 caractères et produit 15 passages, car le découpage tient aussi compte des séparateurs. Le modèle mistral-embed transforme chaque passage en un vecteur de 1 024 dimensions, autrement dit une liste de nombres représentant son sens. La construction se fait par lots de 50. Au total, FAISS contient 14 903 vecteurs pour les 7 586 événements. Chaque passage conserve l'identifiant de l'événement, ses dates, son lieu et son lien.

À montrer / à faire

Distingue la fiche d'exemple et les volumes de tout le corpus. Aucun réindexage en direct.

Transition à dire : « L'index est préparé. Voyons ce qui se passe lorsqu'un utilisateur pose une question. »

Mistral reçoit la question et quatre passages

L'idée à retenir : L'embedding retrouve ; le modèle de chat rédige.

Au moment d'une question, on utilise le même modèle d'embedding que pour les documents. Cela rend leurs vecteurs comparables. FAISS sélectionne les quatre passages les plus proches : c'est le paramètre top-k. Le code assemble ensuite un contexte avec le texte et les métadonnées, puis le transmet avec la question à minstral-8b-2512. Pour la soutenance, ce modèle remplace mistral-small-latest, limité par Mistral aujourd'hui. Le prompt demande de répondre seulement à partir du contexte, de citer les informations disponibles et de ne pas inventer. La température est basse, à 0,1, pour limiter la variation ; la réponse est limitée à 700 tokens. Le JSON renvoie la réponse et les sources récupérées, regroupées par événement. Attention : quatre passages peuvent provenir du même événement. Et l'absence de seuil de pertinence signifie que le modèle doit encore décider si le contexte permet de répondre.

À montrer / à faire

Pointe la séparation embedding/chat et le texte du prompt. Si demandé, ouvre SYSTEM_PROMPT dans rag.py.

Transition à dire : « Ces étapes sont séparées dans le code pour pouvoir les tester et les remplacer. »

La préparation de l'index précède les questions

L'idée à retenir : Deux parcours distincts : construire le catalogue et répondre aux utilisateurs.

L'architecture distingue la préparation des données du traitement des questions. La préparation collecte les fiches, les normalise, calcule leurs embeddings et sauvegarde l'index. Elle est faite avant l'utilisation du chatbot et appelle déjà Mistral pour les vecteurs. À chaque question, l'API appelle le service RAG, qui recharge l'index si nécessaire, recherche les passages et sollicite le modèle de chat. Le retriever et le client Mistral sont initialisés à la première question puis conservés en mémoire pour les suivantes. Le dossier scripts contient les commandes de lancement, tandis que src contient la logique métier. Cette séparation permet d'utiliser le même moteur depuis le terminal, l'API et l'évaluation. Elle évite de relancer la collecte ou l'indexation pour chaque demande.

À montrer / à faire

Si Jérémy demande le code, montre service.py puis rag.py. Évite de naviguer entre toutes les fonctions.

Transition à dire : « L'API donne aux équipes un contrat simple pour tester cette chaîne. »

L'API rend le POC accessible aux équipes

L'idée à retenir : Une question HTTP, une réponse JSON, un environnement conteneurisé.

FastAPI expose trois routes métier. Health indique que le serveur répond et montre sa configuration. Ask accepte une question et renvoie un JSON avec la réponse, les sources et le nombre de passages récupérés. Rebuild reconstruit l'index depuis le fichier Parquet ; il ne lance pas une nouvelle collecte. Swagger, sur docs, permet de lire et tester le contrat sans écrire de client. Docker embarque le code, les dépendances et le seed. Au premier démarrage, l'index et les données sont copiés dans un volume persistant. La clé Mistral est injectée par l'environnement et n'est pas versionnée. Le jeton d'administration de rebuild est optionnel et doit être configuré avant une exposition publique. Le POC reste dépendant d'internet pour vectoriser la question et générer la réponse.

À montrer / à faire

Pendant la démo, ouvre Swagger et montre les routes ; garde /rebuild fermé. Tu peux cliquer le lien Démo de la slide suivante.

Transition à dire : « Je vais maintenant montrer un cas présent dans le corpus et un cas hors périmètre. »

La démonstration montre un résultat et une limite

L'idée à retenir : Un événement parisien sourcé, puis une demande à Lyon hors corpus.

Je commence par montrer que l'API répond et que ses routes sont documentées. Pour le premier cas, je demande des informations sur Concert Fishers à Paris. Le résultat doit permettre de retrouver l'événement, son lieu, sa date et son URL. Il s'agit d'un exemple du corpus livré, pas d'une sortie à venir aujourd'hui. Je compare la réponse au premier événement source. Les autres sources sont les voisins récupérés et ne sont pas toutes nécessaires à cette réponse. Pour le second cas, je demande des expositions photo à Lyon. Le corpus contient Paris. Dans la répétition directe d'aujourd'hui, le modèle dit qu'il ne sait pas et n'invente pas d'exposition lyonnaise. FAISS renvoie quand même des passages : cela illustre la limite de la recherche sans seuil de pertinence.

À montrer / à faire

140 s au total : 20 s health/Swagger ; 50 s cas nominal ; 40 s Lyon ; 30 s explication et retour au deck. Une tentative live maximum en cas de quota, puis enregistrement. Mode secours disponible hors réseau.

Transition à dire : « La démonstration donne un exemple ; les tests et les métriques complètent cette preuve. »

09:55 → 11:00 • 65 secondes • repère incluant gestes et pauses

Les tests du code et la qualité des réponses se complètent

L'idée à retenir : 76 tests réussis aujourd'hui ; quatre cas fonctionnels annotés.

J'ai exécuté aujourd'hui la suite : les 76 tests passent. Ils vérifient notamment les filtres, la normalisation, la construction et le rechargement de l'index, la forme des réponses, la validation API et le calcul des métriques. Ces tests utilisent des doublures pour les services externes. Ils protègent les comportements du code, sans garantir que Mistral est accessible ni que toutes les réponses métier sont bonnes. Un deuxième niveau contient quatre questions annotées avec des réponses et des titres attendus. Les résultats enregistrés classent les quatre cas comme corrects selon les règles locales. Ces règles sont assez permissives : un titre peut être retrouvé dans les sources sans que la réponse soit parfaite. Je complète donc cette évaluation par RAGAS. L'automatisation en CI serait une prochaine étape ; elle n'est pas implémentée ici.

À montrer / à faire

Annonce le résultat des tests, puis explique ce qu'ils couvrent. Montre le petit jeu CSV si demandé.

Transition à dire : « RAGAS nous aide à distinguer une bonne recherche d'une réponse pleinement correcte. »

Les scores RAGAS orientent les améliorations

L'idée à retenir : Trois cas évalués : fidélité élevée, correction encore à améliorer.

L'évaluation RAGAS porte sur trois cas : Concert Fishers, le Salon de la Photo et une sortie en famille avec visite théâtralisée. Pour chaque cas, je conserve la question, les passages réellement transmis au modèle, la réponse et une référence humaine. Un modèle juge calcule quatre métriques. La fidélité mesure si la réponse est soutenue par le contexte. La pertinence mesure son adéquation à la question. La précision du contexte apprécie la pertinence et le classement des passages récupérés ; elle ne mesure pas la couverture de tout le catalogue. La correction compare la réponse à la référence. Les moyennes enregistrées le 23 juillet sont 0,900, 0,846, 1,000 et 0,683. Le dernier score montre que des écarts demeurent même avec un contexte utile. Sur le cas famille, le score de correction est d'environ 0,581 et la réponse ajoute des informations pratiques à vérifier. La priorité est donc de mieux cadrer la réponse, contrôler les faits et élargir le jeu d'évaluation. Trois exemples démontrent la méthode, pas la performance générale.

À montrer / à faire

Lis les quatre scores avec leur sens, puis donne la limite du petit échantillon. En discussion, ouvre les exemples RAGAS du rapport.

Transition à dire : « Ces résultats et la démonstration permettent d'identifier les limites prioritaires. »

Les limites définissent le travail avant production

L'idée à retenir : Actualiser les données, renforcer le refus et mesurer davantage.

La première limite est le corpus : il est limité à Paris et figé en juillet. Pour recommander des sorties actuelles, il faut automatiser son actualisation et filtrer les dates au moment de la demande. La deuxième concerne la recherche : le top-k renvoie les voisins les plus proches même lorsqu'ils ne répondent pas réellement à la question. Le refus repose alors sur le prompt et le modèle. Un seuil calibré et des filtres explicites seraient plus robustes. La troisième limite est l'exploitation. L'index est local, mais les embeddings de question et la génération appellent Mistral. Aujourd'hui, Mistral Small était limité. La démo a été rétablie avec Ministral 8B, sans changer les embeddings. Les erreurs fournisseur sont désormais contrôlées. Ce nouveau modèle doit être réévalué ; les anciens scores ne lui sont pas attribués. Enfin, trois cas RAGAS restent insuffisants pour conclure à une qualité générale.

À montrer / à faire

Relie chaque limite à une amélioration. Si la démo enregistrée a été utilisée, rappelle simplement la cause observée.

Transition à dire : « Je propose donc une suite mesurée, fondée sur les risques identifiés. »

Un pilote mesuré est la prochaine étape

L'idée à retenir : Le POC livre la chaîne complète ; la valeur produit reste à mesurer.

Pour conclure, le POC livre une chaîne complète : des données réelles, un index persistant, une génération augmentée, une API, Docker, des tests et une méthode d'évaluation. Je recommande d'abord d'actualiser le corpus et d'ajouter les garde-fous de ville et de date. Ensuite, d'élargir le jeu annoté et de comparer les paramètres de recherche. Un pilote doit mesurer la réponse utile, les refus corrects, les clics vers les sources, la latence et le coût. Je vous montre brièvement les preuves dans le rapport et le dépôt. Le rapport documente les choix et les résultats. Dans le dépôt, scripts permet de relancer les étapes, src contient le moteur et l'API, tests les validations, et seed-data le corpus de référence. Ces éléments permettent à une autre personne de reproduire et d'examiner le POC. Je suis prêt à répondre à vos questions.

À montrer / à faire

30 s de conclusion ; 20 s rapport (architecture, évaluation, limites) ; 25 s dépôt (scripts/, src/, tests/, seed-data/).
Retourne ensuite au deck. Tous les liens sont dans le guide.

Transition à dire : « Discussion : écoute la question, réponds en une phrase, puis donne la preuve ou la limite utile. »

Guide et diaporama synchronisés sur les mêmes 12 étapes. Les supports précédents sont conservés dans archives/.